

ALGORITMI E STRUTTURE DATI

ANALISI DEGLI ALGORITMI

AULA VIRTUALE DEL 27-04-2026

ANALISI DEGLI ALGORITMI

Abbiamo visto che il tempo di esecuzione di algoritmi ricorsivi può essere espresso tramite relazioni di ricorrenza: il tempo richiesto da una procedura o algoritmo è uguale al tempo speso all'interno della procedura più il tempo speso per le chiamate ricorsive. Ad esempio, la relazione di ricorrenza relativa all'algoritmo di ricerca binaria è T(n) = O(1) + T(n/2). In particolare, nel caso peggiore:

$$T(n) = 2 + T\left(\left\lceil \frac{n-1}{2} \right\rceil\right)$$

ANALISI DEGLI ALGORITMI

Metodo dell'iterazione

Il modo più intuitivo per risolvere una relazione di ricorrenza consiste nello «srotolare» la ricorsione, riducendola ad una sommatoria dipendente solo dalla dimensione n del problema iniziale. In sostanza l'idea centrale è semplice: l'algoritmo richiama se stesso su problemi più piccoli. Per capire quanto tempo impiega in totale, dobbiamo «srotolare» o espandere matematicamente queste chiamate una dentro l'altra, finchè non capiamo qual è lo schema generale e arriviamo alla fine della ricorsione (il caso base).

ANALISI DEGLI ALGORITMI

Metodo dell'iterazione

Partendo dall'equazione di partenza: $T(n) = c + T(\frac{n}{2})$

Ossia, risolvere un problema di dimensione n costa un tempo fisso c, più il tempo necessario per risolvere lo stesso problema grande la metà(n/2) nel caso di ricerca binaria ricorsiva.

Step 1. «Srotolamento»: Sostituiamo iterativamente il termine T a destra con la sua definizione. Sappiamo che se la formula vale per n, vale anche per n/2.

Quindi $T(\frac{n}{2})=c + T(\frac{n}{4})$

ANALISI DEGLI ALGORITMI

Metodo dell'iterazione

Sostituiamo questo blocco nell'equazione principale: T(n) = c + [c+ T($\frac{n}{4}$)] = $2c + T(\frac{n}{4})$.

Facciamo un altro passo per vedere se capiamo lo schema. Quanto vale T($\frac{n}{4}$)?

Vale T($\frac{n}{8}$). Sostituiamo di nuovo: T(n) = 2c + [c+ T($\frac{n}{8}$)] = $3c + T(\frac{n}{8})$.

ANALISI DEGLI ALGORITMI

Metodo dell'iterazione

Guardando i passaggi precedenti, notiamo un legame tra il coefficiente di c ed il denominatore di n. Dopo aver ripetuto l'operazione i volte, l'equazione avrà questa forma generale (individuiamo la regola generale o pattern):

$$T(n) = i * c + [c + T(\frac{n}{2^i})]$$

La domanda è quando trovare il punto di arresto? La ricorsione si ferma quando arriviamo al caso base, $\frac{n}{2^i}$=1 ossia n =2^i ossia il numero totale di passi

i = $log_2 n$ quindi la complessità finale è $T(n)=\Theta(\log n)$

ANALISI DEGLI ALGORITMI

Metodo di sostituzione

Esiste uno stretto legame tra induzione e ricorsione. Il metodo di sostituzione utilizza la ricorsione per risolvere relazioni di ricorrenza. Esso consiste fondamentalmente su due fasi principali: intuire una possibile soluzione e poi usare l'induzione matematica per dimostrare che è corretta.

$T(n) = T(\lfloor n/2 \rfloor)+n$

ANALISI DEGLI ALGORITMI

Metodo di sostituzione

Step 1. Intuizione

Per prima cosa, dobbiamo «indovinare» quale potrebbe essere la complessità asintotica (conta anche l'esperienza). Per esempio: O(n)

Step 2. Impostare l'ipotesi induttiva

Dire per esempio è O(n) significa, matematicamente, affermare che il tempo di esecuzione T(n) non supererà mai una certa costante c moltiplicata per n. Quindi T(n) <= c* n. L'obiettivo è dimostrare che esiste un numero c>0 che rende vera questa affermazione per tutti i valori di n.

ANALISI DEGLI ALGORITMI

Metodo di sostituzione

Step 3. Passo Base. Verificare per n=1:

$T(1) = 1 <= c * 1$

Step 4. Il passo induttivo («la sostituzione»).

Qui è il «core» del metodo. Assumiamo che la nostra ipotesi sia vera per problemi più piccoli di n (come n/2).

$T(n/2) <= c * (n/2)$

ANALISI DEGLI ALGORITMI

Metodo di sostituzione

Sempre Step 4. Dalla nostra equazione di partenza abbiamo:

$T(n) <= c*(n/2) + n$

Raccogliendo la n, abbiamo:

$T(n) <= (\frac{c}{2}+1)\ n$

ANALISI DEGLI ALGORITMI

Metodo di sostituzione

Step 5. Trovare la nostra costante c, ossia dimostrare T(n) <= c* n

Siamo arrivati che T(n) <= ($\frac{c}{2}$+1) n

Affinchè la nostra dimostrazione abbia successo, questa quantità deve essere minore o uguale al nostro obiettivo c*n:

($\frac{c}{2}$+1) n <= c*n

dividendo per n abbiamo $\frac{c}{2}$+1 <= c , risolvendo la disequazione 1<=c/2 ovvero c>= 2. Quindi l'intuizione iniziale era corretta T(n) = O(n).

ANALISI DEGLI ALGORITMI

Il Teorema Fondamentale delle Ricorrenze (conosciuto internazionalmente come *Master Theorem*) è lo strumento principale per analizzare in modo rapido la complessità asintotica degli algoritmi basati sulla tecnica del "divide et impera" (divide and conquer).
Invece di dover sviluppare l'albero della ricorsione passo dopo passo e calcolare somme complesse ogni singola volta, questo teorema ti fornisce una "ricetta" pronta all'uso per una specifica famiglia di relazioni di ricorrenza.

ANALISI DEGLI ALGORITMI

Forma Standard

Il teorema si applica quando il tempo di esecuzione di un algoritmo può essere espresso con un'equazione di questa forma:

$$T(n) = aT(\frac{n}{b}) + f(n)$$

Dove:

n: E’ la dimensione dell’input del problema originale;

a: E’ il numero di sottoproblemi in cui dividi il problema principale (deve essere a >= 1);

ANALISI DEGLI ALGORITMI

b: E’ il fattore di riduzione, ovvero quanto diventa più piccolo l’input in ogni sottoproblema rispetto all’originale (deve essere b >1);

f(n): Rappresenta il costo (in tempo) per dividere il problema iniziale e per ricombinare le soluzioni dei sottoproblemi.

ANALISI DEGLI ALGORITMI

Il cuore del Master Theorem è un confronto. Immagina l'albero della ricorsione: da una parte c'è il costo per dividere e unire (rappresentato da f(n) a livello della radice), dall'altra c'è il costo di base per risolvere i piccolissimi problemi alla fine della ricorsione (le "foglie" dell'albero).

Il numero totale di foglie nell'albero è proporzionale a $n^{log_b a}$. Il teorema confronta proprio la funzione f(n) con questa quantità $n^{log_b a}$. Chi dei due cresce più velocemente "vince" e determina la complessità finale dell'algoritmo.

ANALISI DEGLI ALGORITMI

Confrontando f(n) e $n^{log_b a}$, si presentano tre scenari possibili:

Caso 1: Le foglie dominano (Il peso è in fondo all'albero)

Se f(n) cresce polinomialmente più lentamente del numero delle foglie, significa che il tempo speso per dividere e combinare è trascurabile rispetto al tempo speso per risolvere i casi base.

Condizione: f(n) = O($n^{log_b a - \epsilon}$) per una qualche costante $\epsilon > 0$.

Soluzione: La complessità è dominata dalle foglie: T(n) = $\Theta(n^{log_b a})$

ANALISI DEGLI ALGORITMI

Caso 2: C'è un equilibrio (Il peso è distribuito uniformemente)

Se f(n) cresce esattamente alla stessa velocità del numero di foglie, il lavoro è distribuito in modo equilibrato su tutti i livelli dell'albero della ricorsione.

Condizione: f(n) = O($n^{log_b a}$).

Soluzione: Moltiplichiamo il costo di un livello per il numero di livelli (logaritmico) T(n) = $\Theta(n^{log_b a} \log n)$

ANALISI DEGLI ALGORITMI

Caso 3: La radice domina (Il peso è in cima all'albero)

Se f(n) cresce polinomialmente più velocemente del numero delle foglie, significa che il lavoro grosso viene fatto per dividere l'array o ricombinare i risultati, e i casi base sono trascurabili.

Condizione: f(n) = $\Omega(n^{log_b a+\varepsilon})$ per ε > 0. Deve valere anche una condizione di regolarità, ovvero a f(n/b) <= cf(n) per una costante c<1 e per n sufficientemente grande (significa che i sottoproblemi non «esplodono» in costo).

Soluzione: la complessità è dominata dal costo di divisione/combinazione
T(n) = $\Theta(f(n))$

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

